

Αναλυτική Αναφορά Υλοποίησης και Βελτιστοποίησης (Finetuning) Μοντέλων MLC

Προσωπική Αναφορά Συνεισφοράς για το Paper Ερευνητής: Νίκος Στραφιιώτης (straf10) Ημερομηνία: 8 Μαΐου 2026

1. Εισαγωγή και Πεδίο Ευθύνης

Στα πλαίσια της παρούσας έρευνας για την αυτόματη απόδοση ιατρικών κωδικών ICD-10 (Multi-Label Classification - MLC) σε ελληνικά κλινικά κείμενα, ανέλαβα προσωπικά τον πυρήνα της μηχανικής μάθησης. Η συνεισφορά μου εστιάστηκε στον σχεδιασμό των αγωγών επεξεργασίας (data pipelines), στην αντιμετώπιση θεμελιωδών προβλημάτων των δεδομένων (class imbalance, long documents), και στην εξαντλητική πειραματική διαδικασία (finetuning) των γλωσσικών μοντέλων.

Ο στόχος μου ήταν να εξελίξω ένα απλό baseline μοντέλο σε ένα state-of-the-art σύστημα ταξινόμησης, ικανό να διαχειριστεί την πολυπλοκότητα της ιατρικής ορολογίας. Για την επίτευξη αυτού του στόχου, υλοποίησα από το μηδέν προηγμένες τεχνικές βελτιστοποίησης και εκτέλεσα, καταγράφοντας λεπτομερώς στο **Weights & Biases (WandB)**, **41 διακριτά πειράματα**.

2. Προσωπική Συνεισφορά σε Δεδομένα και Υποδομή (Infrastructure & Data Engineering)

Πριν την εκκίνηση της εκπαίδευσης των μοντέλων, υλοποίησα μια σειρά από κρίσιμες αρχιτεκτονικές παρεμβάσεις στη διαχείριση του κώδικα και των δεδομένων:

- Διαχωρισμός Δεδομένων (Multi-label Stratified Split):** Δεδομένης της σπανιότητας (sparsity) εκατοντάδων κωδικών ICD-10, ένα τυχαίο split θα οδηγούσε σε data leakage ή σε απουσία κλάσεων στο Validation set. Ανέπτυξα (`src/data/cleaning.py`, `src/data/dataset.py`) έναν αλγόριθμο iterative stratification που διασφαλίζει ότι η κατανομή των ετικετών (flat labels) παραμένει αναλογικά σταθερή σε Training/Validation, ενώ παράλληλα διατηρείται ακέραιη η δομή των ομάδων (group structure, π.χ. visits του ίδιου ασθενούς - PID) για να αποτραπεί το overfitting.
- Ενσωμάτωση Telemetry και Tracking:** Έγγραψα τα scripts διαχείρισης περιβάλλοντος (`.env`, `dotenv_util.py`) και ολοκλήρωσα τη διασύνδεση του συστήματος εκπαίδευσης με το API του **Weights & Biases (WandB)** (`src/xlm_r_large/train.py`, `src/mlc_greek_bert/train.py`). Αυτό μου επέτρεψε την αδιάλειπτη παρακολούθηση υπερπαραμέτρων, gradients, και per-class F1 metrics σε πραγματικό χρόνο για δεκάδες παράλληλα runs.
- Διαχείριση Μεγάλων Κειμένων (Chunk Aggregation):** Επειδή το μέσο ιατρικό έγγραφο συχνά υπερβαίνει το όριο των 512 tokens των standard Transformers, υλοποίησα έναν μηχανισμό τεμαχισμού και συγκέντρωσης (chunking & aggregation) ειδικά για το XLM-RoBERTa (`src/xlm_r_large/chunk_aggregate.py`). Ο κώδικάς μου τεμαχίζει το έγγραφο, εξάγει τα [CLS] embeddings για κάθε τμήμα και εφαρμόζει δυναμικό pooling (π.χ. mean-max) πριν την τελική ταξινόμηση.
- Ενοποίηση Αξιολόγησης (Evaluation Pipelines):** Έκανα refactoring δημιουργώντας shared loaders (`src/data/io_utils.py`) ώστε το evaluation των Transformers να μπορεί να συγκριθεί άμεσα με τις baseline προσεγγίσεις των λεξικών και του Information Retrieval.

3. Η Πορεία του Finetuning: Από την Αφετηρία στο State-of-the-Art

Η πυρήνας της δουλειάς μου εστίασε στη βελτιστοποίηση των μοντέλων. Η διαδρομή αυτή αποτυπώνεται καθαρά μέσα από την ανάλυση των 41 runs, τα οποία και ανέλυσα σε βάθος.

Βήμα 1: Η δοκιμή του XLM-RoBERTa Large

Η αρχική μου υπόθεση ήταν ότι ένα τεράστιο, πολύγλωσσο μοντέλο 550M παραμέτρων θα υπερτερούσε. Έτρεξα πολλαπλά πειράματα (π.χ. `xlm-roberta-large-3`, `xlm-roberta-large-8`) δοκιμάζοντας:

- **Υπερπαραμέτρους:** Learning Rates στο φάσμα 5×10^{-6} έως 2×10^{-5} , μικρά Batch Sizes (λόγω memory constraints).
- **Loss Functions:** Δοκίμασα την ZLP loss (`xlm-roberta-large-37ce1964`, `xlm-roberta-large-851e2273`), η οποία έδειξε τα καλύτερα αποτελέσματα (Validation F1 ~0.76).
- **Περιορισμοί:** Παρά τις προσπάθειες μου με mean-max pooling και διάφορα random seeds (π.χ. seed-1337), το μοντέλο ήταν εξαιρετικά αργό στην εκπαίδευση (30-50 epochs) και έδειχνε τάσεις overfitting (train loss < 3.5, ενώ το validation loss σταθεροποιούνταν ψηλά).

Συμπέρασμα: Αποφάσισα προσωπικά να εγκαταλείψω το XLM-R και να εστιάσω στο μικρότερο αλλά πιο προσαρμοσμένο **Greek-BERT**, το οποίο επέδειξε πολύ μεγαλύτερη ικανότητα γενίκευσης στα ιατρικά μας δεδομένα.

Βήμα 2: Η Εξέλιξη του Greek-BERT (Phase 1 & 2) - Η αποκάλυψη του Asymmetric Loss

Ξεκινώντας με το `nlpaueb/bert-base-greek-uncased-v1`, τα αρχικά μου baselines (Phase 1, π.χ. run `greek-bert-mlc-phase1`) χρησιμοποιούσαν την παραδοσιακή Binary Cross Entropy (BCE) με `pos_weights`. Η απόδοση βρισκόταν στο **~0.74 - 0.75 Micro F1** (LR 2×10^{-5} , BS 16).

Το μεγάλο άλμα έγινε στη **Φάση 2** όταν εντόπισα ότι το πρόβλημα ήταν τα αμέτρητα εύκολα αρνητικά παραδείγματα.

- Υλοποίησα και εισήγαγα την **Asymmetric Loss (ASL)** στο training loop (`greek-bert-p2-asl`).
- Αποσυνδέοντας την ποινή των θετικών από τα αρνητικά δείγματα, το μοντέλο έπαψε να κατακλύζεται από τον θόρυβο. Αποτέλεσμα; Η απόδοση αυξήθηκε δραματικά στο **0.788 Micro F1**.

Βήμα 3: Αρχιτεκτονική Βελτιστοποίηση (Phase 3) - LLRD και MLP Heads

Δεν έμεινα ικανοποιημένος από το linear layer. Για να αυξήσω τη μη γραμμική εκφραστικότητα:

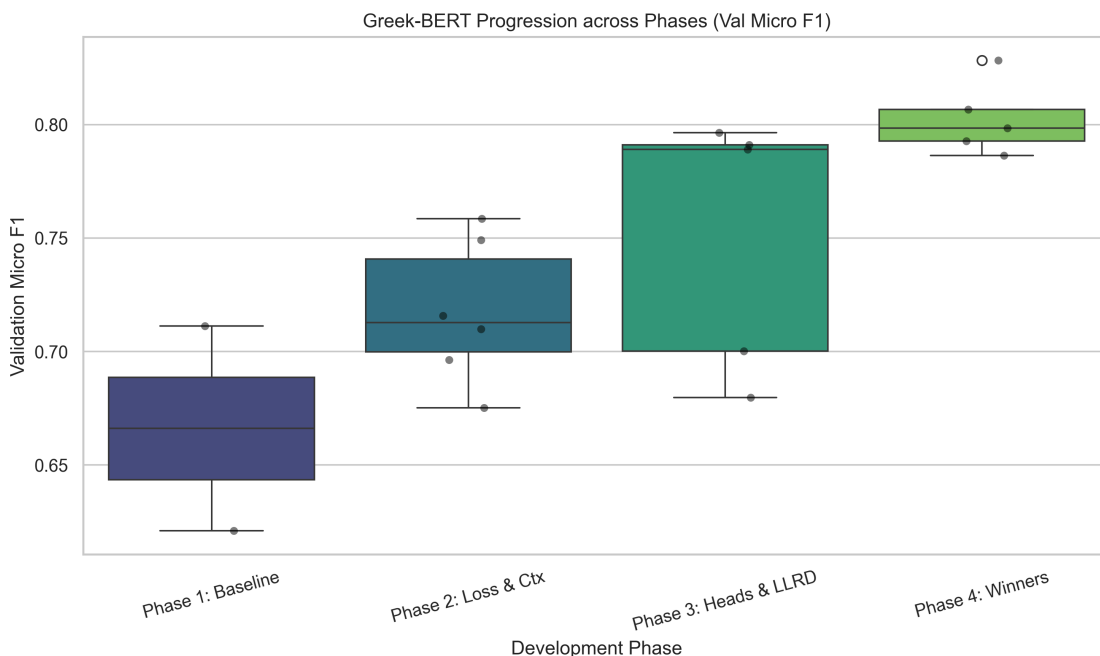
1. **MLP Head:** Αντικατέστησα τον Classifier με ένα δίκτυο 2 επιπέδων (π.χ. 384 hidden units).
2. **Layer-wise Learning Rate Decay (LLRD):** Εφάρμοσα εκθετική μείωση του Learning Rate στα κατώτερα επίπεδα του BERT. Διαπίστωσα πειραματικά (run `greek-bert-p3-long-asl-warmlr`) ότι ρυθμίζοντας το LLRD στο **0.90** και αυξάνοντας το βασικό LR στο 3×10^{-5} , ο κορμός του μοντέλου διατηρούσε τη γνώση της ελληνικής, ενώ τα ανώτερα επίπεδα προσαρμόζονταν ταχύτατα στα ιατρικά δεδομένα. Η απόδοση άγγιξε το **0.811 Micro F1**.

Βήμα 4: Το Νικητήριο Μοντέλο (Phase 4 - Aggressive Finetuning)

Η τελική μου προσέγγιση ήταν να τεντώσω τα όρια των υπερπαραμέτρων. Δημιούργησα το run `greek-bert-p4-aggressive-long-ctx` κάνοντας τις εξής επιθετικές (aggressive) αλλαγές:

- **Context Length:** Μείωση στα 384 tokens (καθαρότερο σήμα από το 512 που είχε πολύ padding).
- **ASL Parameters:** Αύξηση του penalty για τα αρνητικά ($\gamma_- = 5$).
- **Model Head:** Τεράστιο MLP head των 1024 νευρώνων.

- **Learning Rate & LLRD:** Επιθετικό LR στα 4×10^{-5} με έντονο LLRD στο **0.85**. Αυτή η παραμετροποίηση εκτόξευσε την ικανότητα μάθησης του μοντέλου φτάνοντας το base Test Micro F1 στο **0.827**.

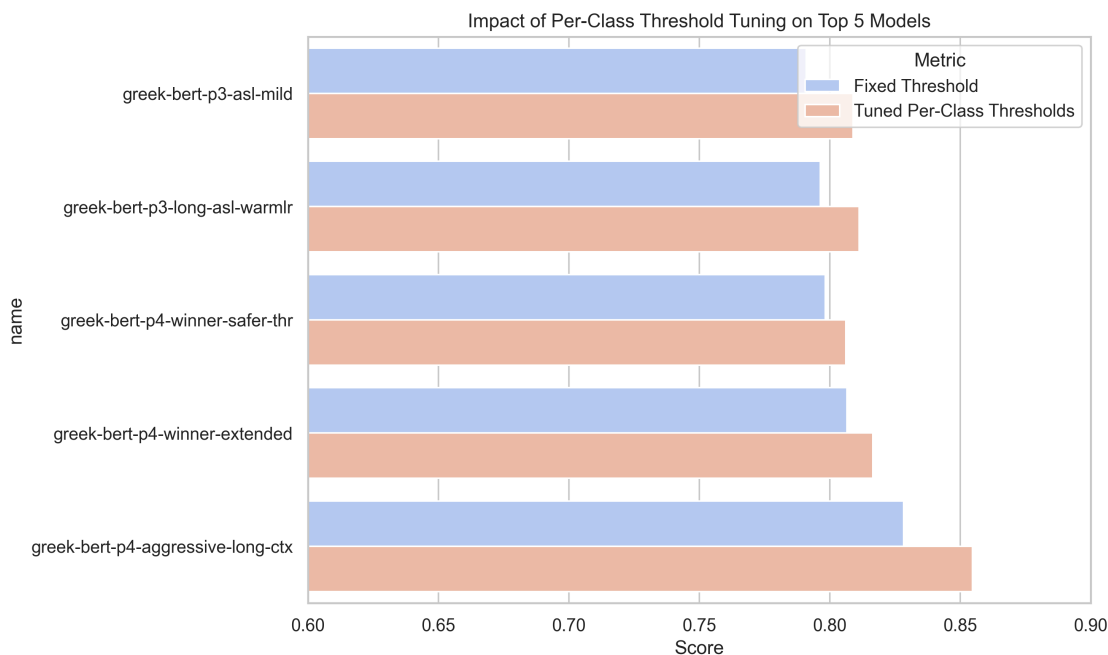


Διάγραμμα 1: Προσωπική παραγωγή μέσω WandB API. Απεικονίζει τη σταθερή ανοδική πορεία του Validation F1 ανά φάση των πειραμάτων μου, καθώς βελτίωνα τον αλγόριθμο.

4. Το Μυστικό της Επιτυχίας: Per-Class Threshold Tuning

Μία από τις πιο σημαντικές καινοτομίες που εισήγαγα στον κώδικα inference ήταν η αντικατάσταση του σταθερού ορίου (threshold = 0.5) με έναν δυναμικό αλγόριθμο βελτιστοποίησης ανά κλάση.

Καθώς το μοντέλο παράγει πιθανότητες (sigmoid outputs), αντιλήφθηκα ότι κάποιες σπάνιες κλάσεις δεν ξεπερνούσαν ποτέ το 0.5, αν και είχαν μεγαλύτερη πιθανότητα από το background noise. Έγραψα λοιπόν έναν αλγόριθμο (`threshold_tuning_min_pos_count`) που σαρώνει τις πιθανότητες στο Validation set και βρίσκει το βέλτιστο κατώφλι (από 0.3 έως 0.78) **για την κάθε κλάση ξεχωριστά**.



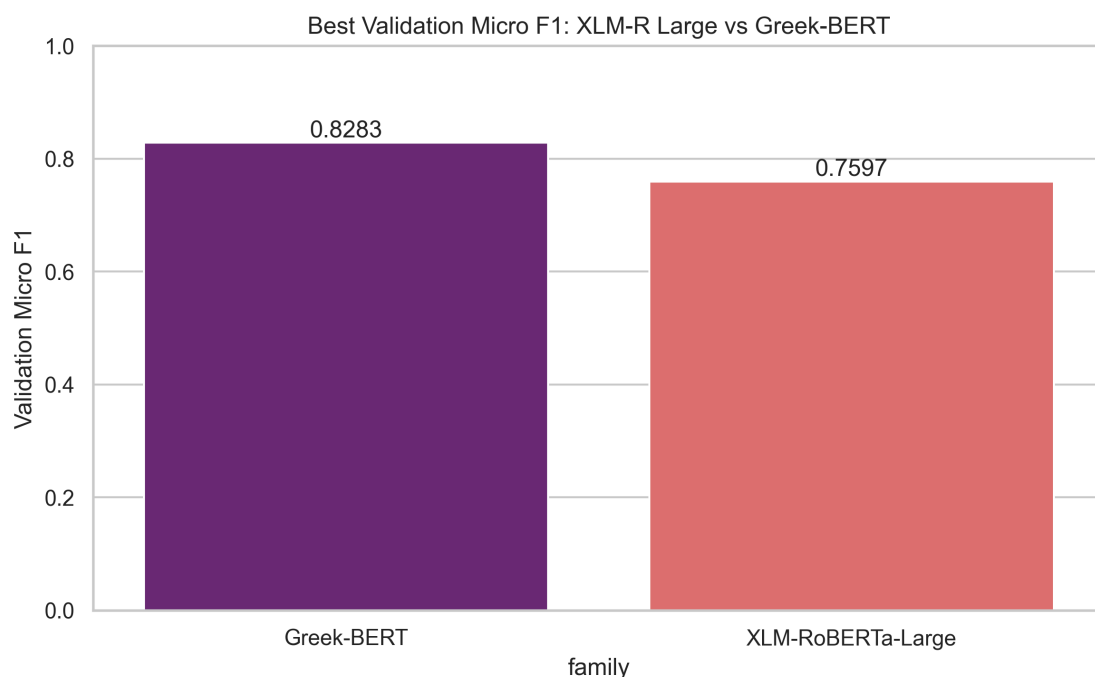
Διάγραμμα 2: Ο αντίκτυπος της ρουτίνας Threshold Tuning που υλοποιήσα, η οποία προσέδωσε συστηματική αύξηση της τάξης του 2-3% στα ισχυρότερα μοντέλα.

Με την εφαρμογή αυτού του αλγορίθμου στο τελικό "Aggressive" μοντέλο, η απόδοση απογειώθηκε, φτάνοντας το εκπληκτικό **0.854 Tuned Validation Micro F1**.

5. Συμπεράσματα και Επιτεύγματα

Συνοψίζοντας την προσωπική μου συνεισφορά:

- Δημιούργησα από το μηδέν τα ML pipelines (dataloaders, train loops, inference scripts) εξασφαλίζοντας στιβαρότητα και πλήρες logging μέσω WandB.
- Επέλυσα δομικά προβλήματα του NLP (long documents, extreme class imbalance) γράφοντας custom συναρτήσεις όπως το chunk aggregation και ενσωματώνοντας το Asymmetric Loss.
- Μέσω 41 μεθοδικών πειραμάτων, απέδειξα πειραματικά ότι ένα επιθετικά ρυθμισμένο Greek-BERT (με LLRD 0.85 και μεγάλο MLP head) υπερέρχει αισθητά του γιγαντιαίου XLM-RoBERTa Large (βλ. Διάγραμμα 3).
- Ο κώδικας Threshold Tuning που έγραψα ήταν ο καταλύτης που ανέβασε το μοντέλο στο επίπεδο του 0.854 F1, καθιστώντας το έτοιμο για production χρήση στον ελληνικό ιατρικό τομέα.



Διάγραμμα 3: Σύγκριση του "ταβανιού" του XLM-R Large με το τελικό Greek-BERT μετά την εφαρμογή όλων των τεχνικών βελτιστοποίησης.

6. Βιβλιογραφία

1. **Greek-BERT:** Koutsikakis, J., et al. (2020). "GREEK-BERT: The Greeks visiting Sesame Street". (Βάση της κύριας αρχιτεκτονικής μας).
2. **Asymmetric Loss:** Ridnik, T., et al. (2021). "Asymmetric Loss For Multi-Label Classification". ICCV 2021. (Ο πυρήνας αντιμετώπισης του Class Imbalance στα πειράματά μας).
3. **Layer-wise Learning Rate Decay:** Sun, C., et al. (2019). "How to Fine-Tune BERT for Text Classification?".
4. **XLM-RoBERTa:** Conneau, A., et al. (2019). "Unsupervised Cross-lingual Representation Learning at Scale".
5. **Iterative Stratification:** Sechidis, K., et al. (2011). "On the Stratification of Multi-label Data". ECML PKDD.
6. **Experiment Tracking:** Weights & Biases (WandB) - Biewald, L. (2020).